

PROTOCOLO SERIAL SKYWATCHER / EQ6

Referencia técnica de comandos, tramas, estados y flujo de comunicación

Basado en el documento de SkyWatcher Motor Controller Command Set y en la implementación actual de INDI SkywatcherAPI / SkywatcherAPIMount. El objetivo es disponer de una referencia práctica para hablar directamente con una EQ6 por UART, analizar tráfico TX/RX y construir un proxy o controlador propio.

Alcance. El documento distingue explícitamente el protocolo del motor controller de las capas superiores (SynScan/INDI). Aquí se documenta el protocolo MC que realmente viaja por el enlace serie hacia las placas de motor.

Importante: el PDF de SkyWatcher contiene el command set completo, mientras que INDI expone solo una parte de ese conjunto en su enum SkywatcherCommand. Cuando hay diferencias, se señalan expresamente.

Fuentes principales

1. SkyWatcher, Skywatcher Motor Controller Command Set — documento aportado en esta conversación.
2. INDI, skywatcherAPI.h — enum de comandos y abstracción de ejes.
<https://github.com/indilib/indi/blob/master/drivers/telescope/skywatcherAPI.h>
3. INDI, skywatcherAPI.cpp — construcción de tramas, lectura de respuestas, reintentos y operaciones de movimiento.
<https://github.com/indilib/indi/blob/master/drivers/telescope/skywatcherAPI.cpp>
4. INDI, skywatcherAPIMount.cpp — uso de la API desde el driver de montura.
<https://github.com/indilib/indi/blob/master/drivers/telescope/skywatcherAPIMount.cpp>

1. Arquitectura de la comunicación en una EQ6

La EQ6 utiliza dos canales lógicos del motor controller: CH1 corresponde al eje RA/Az y CH2 al eje Dec/Alt. El protocolo del documento de SkyWatcher es ASCII y no es un protocolo binario convencional: los comandos se construyen como caracteres ':' + comando + canal + datos + CR.

Elemento	Valor / significado
Velocidad	9600 bit/s
Formato	1 start bit, 8 bits de datos, 1 stop bit, sin paridad (8N1)
Nivel eléctrico	5 V o 3,3 V según hardware
Inicio de comando	':' (0x3A)
Fin de comando	CR = 0x0D
Respuesta normal	'=' + datos + CR
Respuesta de error	!' + código de error + CR
CH1	RA en EQ / Az en Alt-Az
CH2	Dec en EQ / Alt en Alt-Az
Codificación numérica	ASCII hexadecimal, con bytes enviados en orden inverso dentro del campo

En las monturas EQ, el documento indica que TX y RX están separados. El motor controller responde inmediatamente después de recibir y procesar el comando. Esto es importante para tu caso de proxy bidireccional: el tráfico no debe tratarse como una conversación textual continua, sino como una secuencia de transacciones request → response.

La implementación de INDI refuerza este modelo: antes de transmitir hace tcflush(TCIOFLUSH), escribe una trama completa, espera hasta CR (0x0D), valida que la respuesta empiece por '=' o '!' y usa hasta cinco intentos con pausas de 100 ms. El timeout de lectura configurado es de 300 ms.

Trama genérica

: <CMD> <CH> [DATA] \r

Ejemplo: :e1\r

Respuesta normal: =<DATA>\r

Respuesta de error: !<ERR>\r

No hay checksum en el formato descrito. El carácter ':' también tiene una propiedad especial: si el controlador recibe otro ':' antes de CR, abandona el comando parcial y empieza a interpretar el nuevo comando.

2. Codificación de datos: el detalle que más fácilmente provoca errores

Los valores numéricos se expresan con caracteres ASCII '0'-'9' y 'A'-'F'. Sin embargo, para un número de 24 bits los tres bytes no se transmiten en orden big-endian normal. El documento da el ejemplo 0x123456 y especifica la secuencia de caracteres 56 34 12. Para 16 bits, 0x1234 se transmite como 34 12.

Valor lógico	Campo transmitido
8 bit: 0x12	12
16 bit: 0x1234	3412
24 bit: 0x123456	563412

INDI implementa exactamente esta convención mediante Long2BCDstr(): primero emite el byte bajo, después el byte intermedio y finalmente el byte alto, cada uno representado por dos dígitos hex ASCII.

Por tanto, si quieres enviar la posición 0x800012, la trama no contiene '800012' sino '120080' como campo de datos. Esta inversión es especialmente relevante para E, H, M, S, I, U y para las consultas de posición.

3. Tabla completa del command set del documento SkyWatcher

Cmd	Nombre	Datos	Respuesta	Canal	Condición / función
E	Set Position	6 hex	A / X	1/2/3	Fijar posición; motor detenido
F	Initialization Done	ninguno	A / X	1/2/3	Marca inicialización terminada

Cmd	Nombre	Datos	Respuesta	Canal	Condición / función
G	Set Motion Mode	2 bytes de flags	A / X	1/2/3	Selecciona GOTO/Tracking, velocidad y dirección
H	Set Goto Target Increment	6 hex	A / X	1/2	Desplazamiento relativo al destino
M	Set Brake Point Increment	6 hex	A / X	1/2/3	Incremento/punto de frenado
S	Set Goto Target	6 hex	A / X	1/2/3	Fijar destino absoluto
I	Set Step Period (T1)	6 hex	A / X	1/2/3	Periodo del timer T1
T	Set Long Goto Step Period	6 hex	A / X	1/2/3	Periodo para long GOTO
U	Set Brake Steps	6 hex	A / X	1/2/3	Número de pasos de frenado
J	Start Motion	ninguno	A / X	1/2/3	Arranca el movimiento configurado
K	Stop Motion	ninguno	A / X	1/2/3	Parada normal / desacelerada
L	Instant Stop	ninguno	A / X	1/2/3	Parada inmediata; usar con precaución
B	Set Sleep	0/1	A / X	1/2/3	0=WakeUp, 1=Sleep
O	Set Aux Switch On/Off	0/1	A / X	1/2/3	Salida auxiliar
P	Set AutoGuide Speed	0-4	A / X	1/2/3	1x, 0.75x, 0.5x, 0.25x, 0.125x
Q	Run Bootloader Mode	55AA	sin respuesta	—	Entrada al bootloader
V	Set Polar Scope LED brightness	2 hex	A / X	1/2/3	Brillo del LED
a	Inquire Counts Per Revolution	ninguno	B + 6 hex	1/2	CPR / microsteps por revolución
b	Inquire Timer Interrupt Freq	1	B + 6 hex	1	Frecuencia de T1
c	Inquire Brake Steps	ninguno	B + 6 hex	1/2	Pasos de frenado
h	Inquire Goto Target Position	ninguno	B + 6 hex	1/2	Destino actual
i	Inquire Step Period	ninguno	B + 6 hex	1/2	Periodo T1
j	Inquire Position	ninguno	B + 6 hex	1/2	Posición actual
k	Inquire Increment	0/1	B + 6 hex	1/2	Incremento; opción de reset
m	Inquire Brake Point	ninguno	B + 6 hex	1/2	Punto de frenado
f	Inquire Status	ninguno	E + 3 bytes	1/2	Estado de movimiento
g	Inquire High Speed Ratio	ninguno	D + 2 hex	1/2	Multiplicador de alta velocidad
D	Inquire 1X Tracking Period	1	B + 6 hex	1	Periodo 1x
d	Inquire Tele. Axis Position	ninguno	B + 6 hex	1/2	Posición de eje auxiliar / home según implementación
e	Inquire Motor Board Version	ninguno	B + 6 hex	1/2	Firmware de la placa
s	Inquire PEC period	ninguno	B + 6 hex	1/2	Microsteps por revolución de worm
z	Set Debug Flag	1	—	—	Depuración
W	Extended Setting	6 hex	X	—	Ajustes extendidos
q	Extended Inquire	6 hex	X	—	Consultas extendidas
C	Set EEPROM Address	4 hex	—	1	Dirección EEPROM
N	Set EEPROM Value	2 hex	—	1	Valor EEPROM
n	Inquire EEPROM Value	ninguno	—	1	Leer EEPROM
A	Set Register Address	2 hex	—	1/2/3	Dirección de registro
R	Set Register Value	2 hex	—	1/2/3	Valor de registro
r	Inquire Register Value	ninguno	—	1/2/3	Leer registro

La tabla anterior reproduce el command set del documento aportado. No todos estos comandos aparecen en la API pública de INDI actual; los comandos de EEPROM, registros, bootloader, debug y varios extendidos son deliberadamente de bajo nivel y no forman parte del flujo normal del driver.

4. Significado de Set Motion Mode (G)

El comando G es uno de los puntos centrales del protocolo. El documento define varios bits y la implementación de INDI utiliza explícitamente los modos 0, 1, 2 y 3.

Func	INDI	Significado
0	'3' en el código	High-speed Slew mode
1	'1' en el código	Low-speed Slew / tracking mode
2	'2' en GOTO	Low-speed SlewTo / GOTO
3	'0' en GOTO	High-speed SlewTo / GOTO

Para la dirección, INDI usa Direction='0' para sentido positivo/forward y Direction='1' para reverse. Por tanto, una orden directa tiene esta forma:

```
:G1<func><dir>\r
```

Ejemplos: :G10\r = modo 1, dirección forward; :G11\r = modo 1, reverse; :G30\r = modo 3, forward; :G31\r = modo 3, reverse.

El PDF de SkyWatcher describe además bits internos relacionados con GOTO, tracking, slow/fast y coarse/normal. Para un controlador propio conviene seguir la codificación exacta del firmware objetivo y no asumir que todas las combinaciones de bits son equivalentes a los cuatro modos que usa INDI.

5. Flujo de una orden de movimiento

El patrón general documentado por SkyWatcher es: comprobar parada completa → seleccionar modo → cargar parámetros → Start. Para GOTO se monitoriza el estado hasta que el motor se detiene; para Tracking se mantiene el movimiento hasta enviar Stop.

5.1 Slew / tracking continuo

```
:G1<dir>\r      # seleccionar low-speed tracking/slew
:I<periodo>\r    # fijar T1
:J1\r           # iniciar CH1
...
:f1\r           # consultar estado
:K1\r           # parada desacelerada
```

INDI calcula el periodo T1 a partir de la frecuencia del timer y de los microsteps por revolución. Para una velocidad angular ω : $\text{microsteps/s} = \omega \times \text{microsteps/rad}$; $T1 = \text{frecuencia_timer} / \text{microsteps/s}$. En la ruta de alta velocidad INDI divide además por HighSpeedRatio antes de calcular T1.

5.2 GOTO

```
:G1<dir>\r      # modo 1/3 según slew continuo; para GOTO INDI usa 2 o 0
:H1<offset>\r    # offset relativo de 24 bits
:M1<brake>\r     # rampa/punto de frenado
:J1\r           # arrancar
...
:f1\r           # polling de estado
:j1\r           # posición real, si se necesita
```

La implementación actual de INDI usa H (Goto Target Increment) en lugar de S para su función SlewTo. Calcula el target como posición actual + offset, establece el modo 0 (high-speed GOTO) o 2 (low-speed GOTO), ajusta M a un máximo de 3200 microsteps en high speed o 200 en low speed y finalmente envía J.

6. Estado: respuesta de f

El comando f devuelve una respuesta tipo E. El documento define tres bytes de información. INDI interpreta los bits relevantes de la siguiente forma:

Byte / bit	Significado en INDI
Response[1] bit 0	1 = motor running; 0 = stopped
Response[0] bit 0	1 = Slewing (Tracking/Slew); 0 = SlewingTo (GOTO)
Response[0] bit 1	0 = forward; 1 = reverse
Response[0] bit 2	1 = high speed
Response[2] bit 0	1 = inicializado; 0 = no inicializado

Ejemplo conceptual: si f1 devuelve una respuesta cuya representación binaria tiene running=1, el eje está en movimiento. Si además el bit de modo indica SlewingTo, INDI lo considera un GOTO activo. Cuando running pasa a 0, el driver considera el eje parado.

7. Respuestas y códigos de error

Respuesta	Interpretación
=\r	Comando aceptado sin datos
=\r	Comando aceptado; datos devueltos
!0\r	Unknown command
!1\r	Command length error
!2\r	Motor not stopped
!3\r	Invalid character
!4\r	Not initialized
!5\r	Driver sleeping
!7\r	No valid PEC data (documento SkyWatcher)
!8\r	PEC training is running (documento SkyWatcher)

La implementación actual de INDI contiene explícitamente los códigos 0-5. Los códigos adicionales 7 y 8 están presentes en el command-set de SkyWatcher, pero no aparecen en el mapa de errores de este skywatcherAPI.cpp. Eso no significa que el firmware no los produzca.

8. Comandos que INDI usa directamente

Enum INDI	Cmd	Uso
Initialize	F	Inicialización del motor controller
InquireMotorBoardVersion	e	Firmware
InquireGridPerRevolution	a	Microsteps por revolución
InquireTimerInterruptFreq	b	Frecuencia de timer
InquireHighSpeedRatio	g	Ratio de alta velocidad
InquirePECPeiod	s	Microsteps por revolución del worm
InstantAxisStop	L	Parada inmediata
NotInstantAxisStop	K	Parada normal/desacelerada
SetAxisPositionCmd	E	Escribir posición
GetAxisPosition	j	Leer posición
GetAxisStatus	f	Leer estado
SetSnapPort	O	Snap port en modelos compatibles
SetMotionMode	G	Seleccionar modo/dirección
SetGotoTargetIncrement	H	Offset de GOTO
SetBreakPointIncrement	M	Punto de frenado
SetGotoTarget	S	Comando disponible en API, aunque SlewTo actual usa H
SetBreakStep	U	Ramp length de Slew
SetStepPeriod	I	T1
StartMotion	J	Arranque
GetStepPeriod	D	Consulta de periodo; el comentario del header remite al protocolo Merlin
ActivateMotor	B	Sleep/wakeup
SetST4GuideRateCmd	P	Velocidad de autoguiado
GetHomePosition	d	Posición home
SetFeatureCmd	W	Ajustes extendidos
GetFeatureCmd	q	Consultas extendidas
InquireAuxEncoder	d	Encoder auxiliar; mismo carácter que home
SetPolarScopeLED	V	LED del buscador polar

La coincidencia con el documento SkyWatcher es alta, pero hay dos matices importantes: INDI representa algunas funciones con nombres heredados de la API y el carácter 'd' se reutiliza para consultas diferentes según contexto/modelo. Además, el enum no incluye todos los comandos del documento de bajo nivel.

9. Secuencia de inicialización que hace INDI

La implementación actual sigue aproximadamente esta secuencia al abrir la montura:

1. Vacía el buffer serie y ejecuta `CheckIfDCMotor()`. Envía ':' como prueba; si recibe ':' identifica un DC motor.
2. Consulta e mediante :e1\r para obtener la versión de la motor-control board.
3. Extrae MountCode del byte bajo de MCVersion.
4. Consulta q/feature en ambos ejes.
5. Consulta a para microsteps por revolución de CH1 y CH2.
6. Consulta b para la frecuencia del timer de ambos ejes.
7. Consulta g para HighSpeedRatio de ambos ejes.
8. Si no es DC, consulta s para microsteps por revolución del worm.
9. Consulta f para conocer el estado.
10. Si ambos ejes están sin inicializar, lee j1 y j2, guarda las posiciones y ejecuta F1 y F2.

Este orden es útil para reproducir una conexión desde Python y, sobre todo, para comparar un log de tu proxy con lo que hace INDI. El código actual llama a InitializeMC() enviando F por separado a cada canal, aunque existe un comentario antiguo que menciona ':F3'.

10. Cálculo de velocidad

El documento de SkyWatcher define la relación entre velocidad angular, CPR y T1. Para baja velocidad:

```
Speed_CountsPerSec = Speed_DegPerSec * CPR / 360
T1_Preset = TMR_Freq / Speed_CountsPerSec
            = TMR_Freq * 360 / Speed_DegPerSec / CPR
```

Para alta velocidad, si el firmware utiliza N pasos por interrupción:

```
T1_Preset = N * TMR_Freq * 360 / Speed_DegPerSec / CPR
```

INDI implementa la misma idea con radianes: RadiansPerSecond × MicrostepsPerRadian da microsteps/s y divide StepperClockFrequency por esa frecuencia para obtener el periodo de T1. Para alta velocidad utiliza HighSpeedRatio.

El código limita la velocidad solicitada a 500 rad/s y, para T1, impone un mínimo de 6 ticks. Además contiene una corrección de -3 ticks para dos versiones concretas de MCVersion (0x010600 y 0x0010601).

11. Posiciones y offset 0x800000

El documento SkyWatcher añade una regla específica para datos de posición: las posiciones se representan con un offset de 0x800000. Por ejemplo, la posición física/lógica 0x000012 debe prepararse como 0x800012 en el comando, mientras que una lectura 0x801234 representa realmente 0x001234.

Concepto	Valor
Offset protocolario	0x800000
Posición lógica 0x000012	valor transmitido 0x800012
Respuesta 0x801234	posición lógica 0x001234

Este detalle debe mantenerse separado de la inversión de bytes. Son dos transformaciones distintas: primero se aplica la convención de posición y después se serializa el entero de 24 bits como byte bajo → medio → alto.

12. Extended features

El comando q permite consultar características. INDI utiliza q con el identificador 0x01 (GET_FEATURES_CMD) en ambos ejes. Los bits que interpreta actualmente son:

Bit	Característica
0x000001	Dual/Aux encoder disponible
0x000002	PPEC disponible
0x000004	Home indexer disponible
0x000008	EQ/AZ mode
0x000010	PPEC training activo
0x000020	PPEC tracking activo
0x001000	Polar scope LED
0x002000	Common slew start (:J3)
0x004000	Half/full current tracking
0x008000	Wi-Fi

W es el comando de Extended Setting. El documento enumera, entre otros, start/cancel PPEC training, start/cancel PPEC tracking, activar/desactivar encoder, activar corriente completa a baja velocidad, fijar stride, resetear el axis indexer y escribir el buffer RAM en flash.

13. Ejemplos de tramas directas para una EQ6

Las siguientes tramas son ejemplos de estructura. En comandos con datos de 24 bits, debe sustituirse por los seis dígitos hex en orden low-byte → high-byte.

```
# CH1 = RA, CH2 = Dec
:e1\r      # versión MC CH1
:e2\r      # versión MC CH2
```

```

:a1\r      # microsteps/rev CH1
:a2\r      # microsteps/rev CH2
:b1\r      # frecuencia T1 CH1
:b2\r      # frecuencia T1 CH2
:g1\r      # high-speed ratio CH1
:g2\r      # high-speed ratio CH2
:f1\r      # estado CH1
:f2\r      # estado CH2
:j1\r      # posición CH1
:j2\r      # posición CH2
:F1\r      # inicialización CH1
:F2\r      # inicialización CH2
:K1\r      # stop CH1
:L1\r      # instant stop CH1
:J1\r      # start CH1
:G11\r     # modo 1, forward, CH1
:G10\r     # modo 1, reverse, CH1

```

En un programa real no conviene enviar J inmediatamente después de G sin haber configurado el resto de parámetros que el modo seleccionado necesita. La secuencia segura depende de si se pretende tracking, slew manual o GOTO.

14. Cómo implementar el enlace en Python

Para una implementación equivalente al comportamiento de INDI, la capa serie debe hacer tres cosas separadas: (1) construir la trama ASCII, (2) transmitirla completa, y (3) leer exactamente hasta CR y validar el primer carácter de respuesta.

```
def make_cmd(cmd, channel, data=""):
    return f"{{cmd}}{{channel}}{{data}}\r".encode("ascii")

# Ejemplo de consulta:
tx = make_cmd('e', 1)
# b':e1\r'

# Para un entero de 24 bits:
def sw24(value):
    value &= 0xFFFFFF
    return f"{{value & 0xFF:02X}}{((value >> 8) & 0xFF:02X)}{((value >> 16) & 0xFF:02X)}"

# 0x123456 -> '563412'
```

Este fragmento solo muestra la serialización; no sustituye una implementación completa del manejo de timeout, reintentos, estados y sincronización de dos ejes.

Para un proxy bidireccional EQMOD ↔ NEQ6, la propiedad clave es que cada request debe asociarse con su response antes de dejar pasar la siguiente transacción que dependa del resultado. INDI no hace un protocolo de streaming: TalkWithAxis() funciona como una operación síncrona de pregunta/respuesta.

15. Comportamiento de TalkWithAxis() en INDI

Paso	Implementación
Construcción	'.' + comando + canal + datos
Terminación	Añade 0x0D
Reintentos	Máximo 5
Antes de escribir	tcflush(TCIOFLUSH)
Escritura	tty_write_string()
Lectura	hasta 0x0D
Timeout	300000 µs
Respuesta mínima	2 bytes
Respuesta válida	primer byte '=' o 'I'
Respuesta inválida	se ignora y sigue leyendo
Error	se interpreta el primer dígito después de 'I'

Esta implementación tiene una consecuencia importante para un sniffer/proxy: el CR es el delimitador definitivo de una transacción. Si el software recibe bytes parciales, debe acumularlos hasta CR; no debe asumir que una llamada de read() devuelve una trama completa.

16. Diferencia entre protocolo MC y EQMOD / SynScan

No debe confundirse este protocolo con los comandos de alto nivel que un programa de planetario o EQMOD puede manejar. EQMOD puede estar hablando con una capa que a su vez genera estas órdenes hacia las motor boards. Los cuatro ficheros de INDI estudiados aquí implementan directamente la API SkyWatcher MC y convierten operaciones como GOTO, tracking, guiding y stop en comandos del motor controller.

En una conexión EQ6 clásica, el enlace UART físico termina conceptualmente en el motor controller. El master puede ser la hand controller, un adaptador USB/serial, un módulo de interfaz o un software como INDI/EQMOD. El motor controller no recibe RA/Dec en grados directamente: recibe parámetros de posición, modo y periodo de timer.

17. Checklist para capturar y reproducir tráfico

- Configurar 9600 8N1.
- Registrar TX y RX en bytes ASCII y conservar también su representación hexadecimal.
- Separar cada trama por CR (0x0D).
- Etiquetar CH1 y CH2 a partir del carácter inmediatamente posterior al command byte.

- Para datos de 24 bits, reconstruir primero low-byte → mid-byte → high-byte.
- Registrar siempre la respuesta completa, incluyendo '=' o '!' y CR.
- Antes de experimentar con movimiento, consultar e, a, b, g y f en ambos canales.
- Para comandos de escritura, confirmar que la respuesta sea '=' antes de continuar.
- Evitar L (Instant Stop) salvo emergencia: el documento lo describe como parada inmediata.
- Comparar el orden real de comandos con la secuencia InitMount() de INDI.

18. Conclusiones para tu EQ6

El protocolo de la EQ6 no es especialmente complejo una vez separado en sus capas. La capa física es UART 9600 8N1; la capa de framing es ASCII con ':' y CR; el direccionamiento de eje es un único carácter ('1' o '2'); los parámetros son hex ASCII con orden de bytes invertido; y el motor responde inmediatamente con '=' o '!'.
 Lo más relevante para implementar tu propio controlador o proxy es que no necesitas conocer EQMOD para hablar con la motor board: el documento de SkyWatcher y la implementación de INDI proporcionan una especificación funcional suficiente para construir el nivel MC directamente. Lo que sí queda fuera del command set básico son detalles específicos de firmware/modelo y ciertas funciones extendidas.

También queda clara una distinción importante respecto al análisis anterior de tu puente COM4↔COM3: si el objetivo es hacer un proxy transparente, no conviene reinterpretar cada carácter. La unidad de protocolo es comando completo terminado en CR y su correspondiente respuesta terminada en CR. El proxy debe preservar el tráfico y, si necesita inspeccionarlo, decodificarlo en paralelo.

19. Referencias

SkyWatcher Motor Controller Command Set. Documento aportado por el usuario; 5 páginas. El documento especifica el framing ASCII, command set, cálculo de T1 y UART 9600 8N1.

INDI skywatcherAPI.h: enum SkywatcherCommand, estructuras de estado y parámetros internos.
<https://github.com/indilib/indi/blob/master/drivers/telescope/skywatcherAPI.h>

INDI skywatcherAPI.cpp: serialización, TalkWithAxis(), inicialización, Slew(), SlewTo(), Stop y consultas.
<https://github.com/indilib/indi/blob/master/drivers/telescope/skywatcherAPI.cpp>

INDI skywatcherAPIMount.cpp: integración del SkywatcherAPI con el driver de montura, tracking, GOTO, guiding y conexiones.
<https://github.com/indilib/indi/blob/master/drivers/telescope/skywatcherAPIMount.cpp>

Fecha de elaboración: 23 de agosto de 2026. Este documento distingue expresamente entre información del documento SkyWatcher aportado y comportamiento observado en el código actual de INDI.